

HelpDesk Ticket Management System

Test Cases Document — Version 2.0

Stack	ASP.NET Core 8 + Angular 18
Architecture	Clean Architecture — Service Layer, Repository Pattern
PRD Version	2.0 (April 2026)
Total Test Cases	233
Total Modules	13
Document Date	May 2026

154	30	20	29
Positive	Negative	Edge Cases	Security

Table of Contents

Module	Code	Cases
Authentication	AUTH	15
Role-Based Access Control	RBAC	19
Ticket Management	TICKET	26
SLA Engine	SLA	20
Email Notifications	NOTIF	20
Audit Trail	AUDIT	20
Knowledge Base	KB	18
Escalation Workflow	ESC	17
CSAT Survey	CSAT	14
Department Management	DEPT	14
Recurring Ticket Templates	RECUR	15
Reporting & Analytics	REPORT	16
System Administration	ADMIN	19

Covers login/logout flows, JWT validation, account lifecycle emails, and audit logging of auth events.

AUTH-001	Admin login with valid credentials	POSITIVE	High
Steps	1. POST /api/auth/login with {email:'admin@helpdesk.com', password:'Admin@123!'} 2. Verify response 200 OK 3. Check response body contains token, expiry, and role		
Expected	Expected: 200 OK — JWT returned with role=Admin		
AUTH-002	Agent login with valid credentials	POSITIVE	High
Steps	1. POST /api/auth/login with valid agent credentials 2. Verify response 200 OK 3. Confirm role=Agent in response		
Expected	Expected: 200 OK — JWT returned with role=Agent		
AUTH-003	User login with valid credentials	POSITIVE	High
Steps	1. POST /api/auth/login with valid user credentials 2. Verify response 200 OK 3. Confirm role=User in response		
Expected	Expected: 200 OK — JWT returned with role=User		
AUTH-004	Login with incorrect password	NEGATIVE	High
Steps	1. POST /api/auth/login with correct email + wrong password 2. Expect 401 response 3. Verify no token in body		
Expected	Expected: 401 Unauthorized — no token		
AUTH-005	Login with non-existent email	NEGATIVE	High
Steps	1. POST /api/auth/login with email that does not exist 2. Expect 401 response		
Expected	Expected: 401 Unauthorized		
AUTH-006	Login with empty credentials	NEGATIVE	Medium
Steps	1. POST /api/auth/login with empty email and password 2. Expect 400 validation error 3. Check error messages in response		
Expected	Expected: 400 Bad Request — validation errors		

AUTH-007	Login with deactivated account	NEGATIVE	High
Steps	1. Deactivate a user account via Admin 2. Attempt login with that user's credentials 3. Expect 401 response		
Expected	Expected: 401 Unauthorized — account inactive		
AUTH-008	Failed login attempt recorded in audit log	POSITIVE	High
Steps	1. Attempt login with wrong password 2. Query audit log for actor email and action=LOGIN_FAILED 3. Verify IP address captured		
Expected	Expected: Audit entry with action=LOGIN_FAILED and IP address		
AUTH-009	Successful login recorded in audit log	POSITIVE	High
Steps	1. Login successfully 2. Query audit log for action=LOGIN_SUCCESS 3. Verify timestamp and IP address		
Expected	Expected: Audit entry with action=LOGIN_SUCCESS		
AUTH-010	Logout invalidates JWT	POSITIVE	High
Steps	1. Login and receive JWT 2. POST /api/auth/logout with JWT 3. Attempt to access protected endpoint with same JWT 4. Expect 401		
Expected	Expected: 401 on reuse; audit entry LOGOUT recorded		
AUTH-011	Access protected endpoint without JWT	SECURITY	Critical
Steps	1. Call GET /api/tickets with no Authorization header 2. Expect 401 response		
Expected	Expected: 401 Unauthorized		
AUTH-012	Access protected endpoint with expired JWT	SECURITY	Critical
Steps	1. Use an expired JWT in Authorization header 2. Call any protected endpoint 3. Expect 401 response		
Expected	Expected: 401 Unauthorized — token expired		
AUTH-013	Welcome email sent on account creation	POSITIVE	High
Steps	1. Admin creates new user account 2. Check email inbox for welcome email 3. Verify email contains temp password and login link		
Expected	Expected: Welcome email delivered with temp password and link		

AUTH-014	Account deactivation email sent	POSITIVE	Medium
Steps	1. Admin deactivates user account 2. Check that deactivated user receives notification email 3. Verify email is informational (no action link)		
Expected	Expected: Deactivation email sent to affected user		

AUTH-015	Password change logged in audit	POSITIVE	Medium
Steps	1. User changes password via profile 2. Query audit log for action=PASSWORD_CHANGED 3. Verify actor and timestamp recorded		
Expected	Expected: Audit entry PASSWORD_CHANGED with actor details		

RBAC — Role-Based Access Control

19 test cases

Validates that every role permission boundary is enforced — 403 returned for all unauthorized cross-role actions.

RBAC-001	Regular User cannot raise ticket on behalf of another user	SECURITY	Critical
Steps	1. Login as Regular User 2. POST /api/tickets with raisedById = another user's ID 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-002	Admin raises ticket on behalf of user	POSITIVE	High
Steps	1. Login as Admin 2. POST /api/tickets with raisedById = target user ID 3. Expect 201 Created		
Expected	Expected: 201 Created — ticket raised for target user		
RBAC-003	Regular User sees only own tickets	SECURITY	Critical
Steps	1. Login as User A 2. GET /api/tickets 3. Verify all returned tickets have raisedById = User A's ID		
Expected	Expected: Only User A's tickets returned		
RBAC-004	Agent sees only assigned tickets	SECURITY	Critical
Steps	1. Login as Agent B 2. GET /api/tickets 3. Verify all returned tickets have assignedToId = Agent B's ID		
Expected	Expected: Only Agent B's assigned tickets returned		
RBAC-005	Admin sees all tickets	POSITIVE	High
Steps	1. Login as Admin 2. GET /api/tickets 3. Verify tickets from all users and agents returned		
Expected	Expected: All tickets in system returned with pagination		
RBAC-006	Regular User cannot change ticket status	SECURITY	Critical
Steps	1. Login as Regular User 2. PATCH /api/tickets/{id}/status with new status 3. Expect 403		
Expected	Expected: 403 Forbidden		

RBAC-007	Agent cannot change status of unassigned ticket	SECURITY	Critical
Steps	1. Login as Agent 2. PATCH status on ticket not assigned to this agent 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-008	Only Admin can assign tickets	SECURITY	Critical
Steps	1. Login as Agent 2. PATCH /api/tickets/{id}/assign 3. Expect 403 4. Login as Admin and repeat — expect 200		
Expected	Expected: Agent: 403; Admin: 200		
RBAC-009	Only Admin can close tickets	SECURITY	Critical
Steps	1. Login as Agent 2. PATCH /api/tickets/{id}/status body={status:Closed} 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-010	Only Admin can reopen tickets	SECURITY	Critical
Steps	1. Login as Agent 2. PATCH /api/tickets/{id}/status body={status:Reopened} 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-011	Agent cannot access full reports	SECURITY	High
Steps	1. Login as Agent 2. GET /api/reports/ticket-volume 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-012	Agent can access own personal stats	POSITIVE	High
Steps	1. Login as Agent 2. GET /api/reports/my-stats 3. Expect 200 with own data only		
Expected	Expected: 200 OK — personal stats only		
RBAC-013	Regular User cannot access reports	SECURITY	High
Steps	1. Login as User 2. GET /api/reports/any-endpoint 3. Expect 403		
Expected	Expected: 403 Forbidden		

RBAC-014	Only Admin can manage departments	SECURITY	High
Steps	1. Login as Agent 2. POST /api/departments 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-015	Only Admin can manage system settings	SECURITY	High
Steps	1. Login as Agent 2. PUT /api/settings 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-016	Only Admin can create recurring templates	SECURITY	High
Steps	1. Login as Agent 2. POST /api/recurring-templates 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-017	User submits CSAT for own closed ticket	POSITIVE	Medium
Steps	1. Login as User 2. POST /api/csat/{ticketId} for own closed ticket 3. Expect 200		
Expected	Expected: 200 OK — survey submitted		
RBAC-018	Agent cannot submit CSAT	SECURITY	Medium
Steps	1. Login as Agent 2. POST /api/csat/{ticketId} 3. Expect 403		
Expected	Expected: 403 Forbidden		
RBAC-019	Department Head views department tickets (read-only)	POSITIVE	High
Steps	1. Assign user as Department Head of Dept A 2. Login as Department Head 3. GET /api/tickets?department=A — expect success 4. PATCH ticket to change status — expect 403		
Expected	Expected: GET: 200 view-only; PATCH: 403		

TICKET — Ticket Management

26 test cases

Covers full ticket CRUD, status lifecycle transitions, comment flows, validation rules, and audit logging for all ticket mutations.

TKT-001	Create ticket with all required fields	POSITIVE	High
Steps	1. Login as Regular User 2. POST /api/tickets with title, description, categoryId, priority, departmentId 3. Expect 201 Created 4. Verify ticket in DB with status=Open		
Expected	Expected: 201 Created — ticket with status=Open and ticket ID		
TKT-002	Create ticket without Title	NEGATIVE	High
Steps	1. POST /api/tickets with title omitted 2. Expect 400 Bad Request 3. Verify error message mentions 'title'		
Expected	Expected: 400 Bad Request — title is required		
TKT-003	Create ticket without Description	NEGATIVE	High
Steps	1. POST /api/tickets with description omitted 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — description is required		
TKT-004	Create ticket without Category	NEGATIVE	High
Steps	1. POST /api/tickets with categoryId omitted 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — category is required		
TKT-005	Create ticket without Priority	NEGATIVE	High
Steps	1. POST /api/tickets with priority omitted 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — priority is required		
TKT-006	Create ticket without Department	NEGATIVE	High
Steps	1. POST /api/tickets with departmentId omitted 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — department is required		
TKT-007	Title exceeds 150 characters	NEGATIVE	Medium
Steps	1. POST /api/tickets with title = 151-char string 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — title max 150 chars		

TKT-008	Description exceeds 2000 characters	NEGATIVE	Medium
Steps	1. POST /api/tickets with description = 2001-char string 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — description max 2000 chars		
TKT-009	New ticket defaults to Open status	POSITIVE	High
Steps	1. Create any valid ticket 2. Fetch ticket by ID 3. Check status field		
Expected	Expected: status = Open		
TKT-010	Admin assigns ticket to Agent	POSITIVE	High
Steps	1. Login as Admin 2. PATCH /api/tickets/{id}/assign with agentId 3. Expect 200 OK 4. Verify assignedToId updated in DB		
Expected	Expected: 200 OK — ticket assigned, notification sent		
TKT-011	Agent transitions Open to InProgress	POSITIVE	High
Steps	1. Login as assigned Agent 2. PATCH /api/tickets/{id}/status body={status:InProgress} 3. Expect 200 OK		
Expected	Expected: 200 OK — status=InProgress		
TKT-012	Agent transitions InProgress to OnHold	POSITIVE	High
Steps	1. Ticket in InProgress state 2. PATCH status to OnHold 3. Expect 200		
Expected	Expected: 200 OK — status=OnHold, SLA timer paused		
TKT-013	Agent transitions OnHold to InProgress	POSITIVE	High
Steps	1. Ticket in OnHold state 2. PATCH status to InProgress 3. Expect 200		
Expected	Expected: 200 OK — status=InProgress, SLA timer resumed		
TKT-014	Agent transitions InProgress to Resolved	POSITIVE	High
Steps	1. Ticket in InProgress state 2. PATCH status to Resolved 3. Expect 200		
Expected	Expected: 200 OK — status=Resolved		

TKT-015	Admin closes a Resolved ticket	POSITIVE	High
Steps	1. Login as Admin 2. PATCH /api/tickets/{id}/status body={status:Closed} 3. Expect 200		
Expected	Expected: 200 OK — status=Closed, SLA compliance recorded		
TKT-016	Admin reopens a Closed ticket	POSITIVE	High
Steps	1. Login as Admin 2. PATCH /api/tickets/{id}/status body={status:Reopened} 3. Expect 200 4. Verify SLA timer reset		
Expected	Expected: 200 OK — status=Reopened, SLA reset		
TKT-017	Invalid status transition rejected	NEGATIVE	High
Steps	1. Ticket in Open state 2. PATCH status directly to Closed (skipping Resolved) 3. Expect 422 or 400		
Expected	Expected: 400/422 — invalid transition		
TKT-018	User adds comment on own ticket	POSITIVE	Medium
Steps	1. Login as User 2. POST /api/tickets/{id}/comments with own ticket ID 3. Expect 201		
Expected	Expected: 201 Created — comment saved		
TKT-019	User cannot comment on another user's ticket	SECURITY	High
Steps	1. Login as User A 2. POST /api/tickets/{B's ticket id}/comments 3. Expect 403		
Expected	Expected: 403 Forbidden		
TKT-020	Agent comments on assigned ticket	POSITIVE	Medium
Steps	1. Login as Agent 2. POST /api/tickets/{assigned id}/comments 3. Expect 201		
Expected	Expected: 201 Created		
TKT-021	Admin comments on any ticket	POSITIVE	Medium
Steps	1. Login as Admin 2. POST /api/tickets/{any id}/comments 3. Expect 201		
Expected	Expected: 201 Created		

TKT-022	Ticket with optional AffectedAsset saved	POSITIVE	Low
Steps	1. POST /api/tickets with affectedAsset='Laptop SN-2341' 2. Fetch ticket 3. Verify affectedAsset field returned		
Expected	Expected: AffectedAsset stored and returned		
TKT-023	Ticket linked to existing related ticket	POSITIVE	Medium
Steps	1. Create ticket A 2. Create ticket B with relatedTicketId = A's ID 3. Expect 201 4. Verify relatedTicket in response		
Expected	Expected: 201 Created — related ticket linked		
TKT-024	RelatedTicketId pointing to non-existent ticket	NEGATIVE	Medium
Steps	1. POST /api/tickets with relatedTicketId = 99999 (non-existent) 2. Expect 404 or 400		
Expected	Expected: 404 Not Found / 400 Bad Request		
TKT-025	Paginated ticket list returns correct data	POSITIVE	High
Steps	1. Create 25 tickets 2. GET /api/tickets?page=1&pageSize=10 3. Verify 10 items returned and total=25		
Expected	Expected: 10 items, totalCount=25, correct pagination metadata		
TKT-026	All ticket mutations logged in audit trail	POSITIVE	High
Steps	1. Create, assign, update status, add comment on a ticket 2. Query audit log for that ticket's entityId 3. Verify all actions recorded with old/new values		
Expected	Expected: Full audit trail for every mutation		

Validates SLA deadline calculation, timer pause/resume, priority-change recalculation, breach detection, and all edge cases including holidays and On Hold cross-midnight.

SLA-001	SLA deadline auto-set on ticket creation	POSITIVE	Critical
Steps	1. Create a Critical priority ticket 2. Fetch ticket detail 3. Check slaDeadline is set to ~4 business hours from creation		
Expected	Expected: slaDeadline populated correctly		
SLA-002	Critical SLA = 4 hours resolution	POSITIVE	Critical
Steps	1. Create Critical ticket during business hours 2. Verify slaDeadline = createdAt + 4 business hours		
Expected	Expected: Deadline = +4 business hours		
SLA-003	High SLA = 8 hours resolution	POSITIVE	High
Steps	1. Create High priority ticket 2. Verify slaDeadline = createdAt + 8 business hours		
Expected	Expected: Deadline = +8 business hours		
SLA-004	Medium SLA = 3 business days	POSITIVE	High
Steps	1. Create Medium priority ticket 2. Verify slaDeadline = createdAt + 3 business days		
Expected	Expected: Deadline = +3 business days		
SLA-005	Low SLA = 5 business days	POSITIVE	Medium
Steps	1. Create Low priority ticket 2. Verify slaDeadline = createdAt + 5 business days		
Expected	Expected: Deadline = +5 business days		
SLA-006	SLA timer pauses on OnHold	POSITIVE	High
Steps	1. Create ticket, move to InProgress 2. Record elapsed time 3. Move to OnHold 4. Wait 1 hour 5. Verify elapsed time did not increase		
Expected	Expected: SLA timer frozen while OnHold		

SLA-007	SLA timer resumes on InProgress	POSITIVE	High
Steps	1. Ticket in OnHold 2. Move back to InProgress 3. Verify timer resumes from where it paused		
Expected	Expected: Timer resumes correctly, deadline adjusted		
SLA-008	Priority change recalculates SLA from change time	EDGE CASE	Critical
Steps	1. Create Medium ticket (3-day SLA) 2. Wait 1 hour, upgrade to High (8-hour SLA) 3. Verify new deadline = priority change time + 8 hours (not creation time)		
Expected	Expected: SLA deadline = priority-change timestamp + 8 hours		
SLA-009	Ticket created outside business hours	EDGE CASE	High
Steps	1. Create ticket at 9 PM on Friday 2. Verify SLA timer starts at Monday 9:00 AM (business open) 3. Check deadline calculated from Monday morning		
Expected	Expected: SLA timer starts at next business day open		
SLA-010	SLA status = Warning at 75% elapsed	POSITIVE	High
Steps	1. Create Critical ticket (4-hour SLA) 2. Fast-forward 3 hours (75%) 3. Check slaStatus field		
Expected	Expected: slaStatus = 'Warning'		
SLA-011	SLA status = Breached when time exceeded	POSITIVE	Critical
Steps	1. Create ticket 2. Allow SLA time to fully elapse without closure 3. Check slaStatus field		
Expected	Expected: slaStatus = 'Breached'		
SLA-012	SLA warning notification at 75% elapsed	POSITIVE	High
Steps	1. Create High ticket 2. At 6-hour mark (75% of 8h) 3. Verify Admin and Agent received warning email		
Expected	Expected: Warning email sent to Admin and assigned Agent		
SLA-013	SLA breach notification sent to all parties	POSITIVE	Critical
Steps	1. Allow ticket to breach SLA 2. Verify Admin, Agent, and User received breach notification email		
Expected	Expected: Breach email sent to Admin, Agent, and User		

SLA-014	Reopening ticket resets SLA to zero	EDGE CASE	High
Steps	1. Close a ticket 2. Admin reopens ticket 3. Verify slaDeadline = reopen time + SLA for current priority		
Expected	Expected: SLA timer reset from zero on reopen		
SLA-015	Admin manually overrides SLA deadline	POSITIVE	Medium
Steps	1. Login as Admin 2. PATCH /api/tickets/{id}/sla-override with new deadline 3. Expect 200 4. Verify new deadline saved		
Expected	Expected: 200 OK — SLA deadline updated		
SLA-016	Manual SLA override recorded in audit	POSITIVE	High
Steps	1. Admin overrides SLA deadline 2. Query audit log 3. Verify action=SLA_OVERRIDE_MANUAL with old/new values and admin's notes		
Expected	Expected: Audit entry with SLA_OVERRIDE_MANUAL		
SLA-017	Closure records SLA compliance status	POSITIVE	High
Steps	1. Close ticket before SLA breach 2. Fetch closed ticket 3. Verify closedWithinSla = true (immutable)		
Expected	Expected: closedWithinSla field set and immutable		
SLA-018	SLA timer does not count holiday hours	EDGE CASE	High
Steps	1. Admin adds a public holiday 2. Create ticket the day before holiday 3. Verify holiday hours excluded from SLA calculation		
Expected	Expected: Deadline skips holiday hours correctly		
SLA-019	OnHold ticket crossing midnight accrues no overnight time	EDGE CASE	High
Steps	1. Move ticket to OnHold at 11:30 PM 2. Check elapsed SLA the next business morning 3. Verify zero overnight hours accrued		
Expected	Expected: No SLA time accrued overnight during OnHold		
SLA-020	SLA config changes apply to new tickets only	EDGE CASE	High
Steps	1. Change High priority resolution from 8h to 6h in settings 2. Verify existing open High tickets still show 8h SLA 3. Create new High ticket — verify 6h SLA applied		
Expected	Expected: Existing tickets: 8h unchanged; New ticket: 6h applied		

NOTIF — Email Notifications

20 test cases

Validates all notification triggers, async delivery, retry logic, user preference management, and mandatory admin notifications.

NTF-001	User receives confirmation email on ticket creation	POSITIVE	High
Steps	1. User creates ticket 2. Check user's inbox 3. Verify email contains ticket ID, summary, and deep-link		
Expected	Expected: Confirmation email with ticket ID and link		
NTF-002	Admin receives alert on ticket creation	POSITIVE	High
Steps	1. User creates ticket 2. Check admin inbox 3. Verify alert email contains ticket summary		
Expected	Expected: Alert email received by Admin		
NTF-003	Agent and User notified on ticket assignment	POSITIVE	High
Steps	1. Admin assigns ticket to Agent 2. Check Agent inbox — expect assignment notification with ticket link 3. Check User inbox — expect notification with agent name		
Expected	Expected: Both Agent and User receive assignment email		
NTF-004	User notified on every status change	POSITIVE	High
Steps	1. Agent changes ticket status from Open to InProgress 2. Check User inbox 3. Repeat for OnHold and Resolved transitions		
Expected	Expected: Status change email sent to User for each transition		
NTF-005	All parties notified on new comment; includes first 200 chars	POSITIVE	Medium
Steps	1. Agent posts comment (250 chars) 2. Check all party inboxes 3. Verify comment preview = first 200 chars only		
Expected	Expected: All parties emailed; preview truncated to 200 chars		
NTF-006	User notified on ticket close with resolution summary	POSITIVE	High
Steps	1. Admin closes ticket 2. Check User inbox 3. Verify email contains resolution summary if present		
Expected	Expected: Closure email with resolution summary		

NTF-007	Agent and Admin notified on ticket reopen	POSITIVE	High
Steps	1. Admin reopens closed ticket 2. Check assigned Agent and Admin inboxes 3. Verify email states reason for reopening		
Expected	Expected: Reopen notification with reason sent to Agent and Admin		
NTF-008	Admin and Department Head notified on escalation	POSITIVE	Critical
Steps	1. Agent escalates ticket 2. Check Admin inbox — marked urgent 3. Check Department Head inbox — marked urgent with reason		
Expected	Expected: Urgent escalation email to Admin and Dept Head		
NTF-009	All three parties notified on agent reassignment	POSITIVE	High
Steps	1. Admin reassigns ticket from Agent A to Agent B 2. Check Agent A, Agent B, and User inboxes		
Expected	Expected: All three receive reassignment notification		
NTF-010	Email sending is asynchronous	POSITIVE	Critical
Steps	1. Create ticket and measure API response time 2. Verify response returns before email is delivered 3. Email should arrive shortly after but not block response		
Expected	Expected: API responds in <200ms; email arrives asynchronously		
NTF-011	Failed email retried 3 times with exponential backoff	POSITIVE	High
Steps	1. Simulate SMTP failure 2. Verify retry at 30s, then 2 min, then 10 min 3. Verify after 3 failures the attempt is marked as permanently failed		
Expected	Expected: 3 retry attempts at 30s/2m/10m, then failure logged		
NTF-012	All send attempts logged with outcome	POSITIVE	High
Steps	1. Trigger email notification 2. Check email send log 3. Verify entries for success: timestamp, recipient, status=Sent 4. Simulate failure — verify status=Failed logged		
Expected	Expected: All attempts (success/fail) logged with timestamp		
NTF-013	User disables individual notification type	POSITIVE	Medium
Steps	1. User navigates to notification preferences 2. Disables 'Comment Posted' notification 3. Add comment to user's ticket 4. Verify user does NOT receive email		
Expected	Expected: User not notified for disabled notification type		

NTF-014	Admin cannot disable SLA breach notifications	SECURITY	Critical
Steps	1. Admin navigates to notification preferences 2. Attempt to disable SLA breach notification 3. Verify toggle is locked/disabled		
Expected	Expected: SLA breach notification not disableable for Admin		
NTF-015	Admin cannot disable escalation notifications	SECURITY	Critical
Steps	1. Admin attempts to disable escalation alerts in preferences 2. Verify option is locked		
Expected	Expected: Escalation alert always on for Admin		
NTF-016	Email does not exceed 100kB	POSITIVE	Medium
Steps	1. Trigger the most content-heavy email (e.g. comment notification) 2. Check email size 3. Verify < 100kB		
Expected	Expected: Email size < 100kB		
NTF-017	Email contains plain-text fallback	POSITIVE	Medium
Steps	1. Receive any notification email 2. Inspect MIME parts 3. Verify text/plain alternative present		
Expected	Expected: Plain-text MIME part present in all emails		
NTF-018	Dept Head only gets escalation notifs for own dept	POSITIVE	High
Steps	1. Escalate ticket from Dept A 2. Escalate ticket from Dept B 3. Dept Head of A should receive only Dept A escalation		
Expected	Expected: Dept-scoped escalation notification		
NTF-019	CSAT survey email sent after ticket closure	POSITIVE	High
Steps	1. Admin closes ticket 2. Check User inbox 3. Verify survey link in email 4. Verify link expires after 7 days		
Expected	Expected: Survey email with expiring link		
NTF-020	User who opted out does not receive survey email	POSITIVE	Medium
Steps	1. User opts out of CSAT emails in preferences 2. Admin closes user's ticket 3. Verify user does NOT receive survey email		
Expected	Expected: No survey email for opted-out user		

Validates completeness, immutability, and read-only enforcement of the append-only audit log across all event types.

AUD-001	Ticket creation logged	POSITIVE	High
Steps	1. Create ticket 2. Query audit log filtered by entityId=ticket ID 3. Verify action=TICKET_CREATED, actorEmail, timestamp, entityType=Ticket		
Expected	Expected: Audit entry TICKET_CREATED with full actor details		
AUD-002	Ticket status change logged with old/new values	POSITIVE	High
Steps	1. Change ticket status from Open to InProgress 2. Query audit log 3. Verify previousValue=Open, newValue=InProgress		
Expected	Expected: Audit entry TICKET_STATUS_CHANGED with old/new values		
AUD-003	Ticket assignment logged	POSITIVE	High
Steps	1. Admin assigns ticket to Agent 2. Query audit log 3. Verify action=TICKET_ASSIGNED, newValue=agent name/ID		
Expected	Expected: Audit entry TICKET_ASSIGNED with agent info		
AUD-004	Priority change logged with old/new values	POSITIVE	High
Steps	1. Change ticket priority 2. Query audit log 3. Verify previousValue and newValue contain priority labels		
Expected	Expected: Audit entry TICKET_PRIORITY_CHANGED		
AUD-005	Comment addition logged	POSITIVE	Medium
Steps	1. Add comment to ticket 2. Query audit log 3. Verify action=COMMENT_ADDED with entityType=Ticket		
Expected	Expected: Audit entry COMMENT_ADDED		
AUD-006	User account creation logged	POSITIVE	High
Steps	1. Admin creates new user 2. Query audit log 3. Verify action=USER_CREATED with new user email		
Expected	Expected: Audit entry USER_CREATED		

AUD-007	User deactivation logged	POSITIVE	High
Steps	1. Admin deactivates user 2. Query audit log 3. Verify action=USER_DEACTIVATED with actor and target		
Expected	Expected: Audit entry USER_DEACTIVATED		
AUD-008	Role change logged with old and new role	POSITIVE	High
Steps	1. Admin changes user role from User to Agent 2. Query audit log 3. Verify previousValue=User, newValue=Agent		
Expected	Expected: Audit entry USER_ROLE_CHANGED with old/new		
AUD-009	Login success logged with IP	POSITIVE	High
Steps	1. Login successfully 2. Query audit log 3. Verify action=LOGIN_SUCCESS with IP address captured		
Expected	Expected: Audit entry with IP address		
AUD-010	Login failure logged with IP	POSITIVE	High
Steps	1. Attempt login with wrong password 2. Query audit log 3. Verify action=LOGIN_FAILED with IP address		
Expected	Expected: Audit entry LOGIN_FAILED with IP		
AUD-011	Audit log records are read-only — no Update	SECURITY	Critical
Steps	1. Attempt PUT /api/audit-logs/{id} with modified data 2. Expect 405 Method Not Allowed		
Expected	Expected: 405 Method Not Allowed		
AUD-012	Audit log records are read-only — no Delete	SECURITY	Critical
Steps	1. Attempt DELETE /api/audit-logs/{id} 2. Expect 405 Method Not Allowed		
Expected	Expected: 405 Method Not Allowed		
AUD-013	Admin filters audit log by date range	POSITIVE	High
Steps	1. GET /api/audit-logs?from=2025-01-01&to;=2025-01-31 2. Verify all returned entries fall within date range		
Expected	Expected: Filtered results within date range only		
AUD-014	Admin filters audit log by action type	POSITIVE	Medium
Steps	1. GET /api/audit-logs?actionType=TICKET_CREATED 2. Verify all returned entries have action=TICKET_CREATED		
Expected	Expected: Only TICKET_CREATED entries returned		

AUD-015	Admin exports audit log as CSV	POSITIVE	Medium
Steps	1. GET /api/audit-logs/export?from=...&to;=... 2. Verify response is CSV with UTF-8 encoding and header row		
Expected	Expected: Valid CSV file downloaded		
AUD-016	SLA manual override logged with notes	POSITIVE	High
Steps	1. Admin overrides SLA deadline with reason notes 2. Query audit log 3. Verify action=SLA_OVERRIDE_MANUAL, additionalNotes contains reason		
Expected	Expected: Audit entry with override reason in notes		
AUD-017	System settings change logged	POSITIVE	High
Steps	1. Admin updates SLA target for Critical priority 2. Query audit log 3. Verify action=SETTINGS_CHANGED, previousValue and newValue		
Expected	Expected: Audit entry SETTINGS_CHANGED		
AUD-018	KB article events logged	POSITIVE	Medium
Steps	1. Create, publish, and delete a KB article 2. Query audit log 3. Verify KB_ARTICLE_CREATED, KB_ARTICLE_PUBLISHED, KB_ARTICLE_DELETED entries		
Expected	Expected: All KB events logged		
AUD-019	Report generation logged with filters	POSITIVE	Medium
Steps	1. Generate Ticket Volume Report with date filter 2. Query audit log 3. Verify action=REPORT_GENERATED with filter parameters in notes		
Expected	Expected: Audit entry REPORT_GENERATED with parameters		
AUD-020	CSAT survey submission logged	POSITIVE	Medium
Steps	1. User submits CSAT survey 2. Query audit log 3. Verify action=CSAT_SUBMITTED with entityId=ticketId		
Expected	Expected: Audit entry CSAT_SUBMITTED		

Covers article creation, draft/publish lifecycle, version history, search relevance, ticket integration, and per-role access restrictions.

KB-001	Admin creates and publishes KB article	POSITIVE	High
Steps	1. Login as Admin 2. POST /api/kb/articles with title, category, content, tags, status=Published 3. Expect 201 Created 4. Fetch article — verify status=Published and visible to all		
Expected	Expected: 201 Created — article published and visible		
KB-002	Agent creates KB article in Draft status	POSITIVE	High
Steps	1. Login as Agent 2. POST /api/kb/articles with status=Draft 3. Expect 201 Created 4. Verify article NOT visible to Regular Users		
Expected	Expected: 201 Created — article in Draft, hidden from users		
KB-003	Regular User cannot create KB article	SECURITY	High
Steps	1. Login as Regular User 2. POST /api/kb/articles 3. Expect 403		
Expected	Expected: 403 Forbidden		
KB-004	Regular User cannot view Draft articles	SECURITY	Critical
Steps	1. Create Draft article 2. Login as Regular User 3. GET /api/kb/articles/{id} (Draft) 4. Expect 403 or 404		
Expected	Expected: Draft not visible to Regular Users		
KB-005	Regular User views Published articles	POSITIVE	High
Steps	1. Publish a KB article 2. Login as Regular User 3. GET /api/kb/articles/{id} 4. Expect 200 OK		
Expected	Expected: 200 OK — article content returned		
KB-006	Agent edits own Draft article	POSITIVE	Medium
Steps	1. Agent creates Draft article 2. Same Agent calls PUT /api/kb/articles/{id} 3. Expect 200 OK		
Expected	Expected: 200 OK — article updated, new version created		

KB-007	Agent cannot edit another agent's Draft	SECURITY	High
Steps	1. Agent A creates Draft 2. Login as Agent B 3. PUT /api/kb/articles/{Agent A's draft id} 4. Expect 403		
Expected	Expected: 403 Forbidden		
KB-008	Every save creates new version entry	POSITIVE	Medium
Steps	1. Create article 2. Save 3 times with different content 3. GET /api/kb/articles/{id}/versions 4. Verify 3 version entries exist		
Expected	Expected: Version history with 3 entries, each auto-incremented		
KB-009	Admin restores previous version	POSITIVE	Medium
Steps	1. Article has 3 versions 2. Admin calls POST /api/kb/articles/{id}/restore/{versionId} 3. Verify new version 4 created with old content 4. Verify versions 1-3 still intact		
Expected	Expected: Restore creates new version, history preserved		
KB-010	KB search by keyword returns ranked results	POSITIVE	High
Steps	1. Create 3 articles with keyword in title, content, and tags respectively 2. GET /api/kb/articles/search?q=keyword 3. Verify title-match article ranks first		
Expected	Expected: Title match ranked highest in results		
KB-011	KB search across title, content, and tags	POSITIVE	High
Steps	1. Article 1 has keyword in title only 2. Article 2 has keyword in content only 3. Article 3 has keyword in tags only 4. Search for keyword 5. Verify all 3 returned		
Expected	Expected: All 3 articles in search results		
KB-012	User submits helpful/not helpful feedback	POSITIVE	Low
Steps	1. Regular User reads published article 2. POST /api/kb/articles/{id}/feedback body={helpful:true} 3. Expect 200 4. Verify helpfulCount incremented		
Expected	Expected: 200 OK — feedback recorded		

KB-013	View count increments on article open	POSITIVE	Low
Steps	1. Note current viewCount 2. Regular User fetches article 3. Verify viewCount incremented by 1		
Expected	Expected: viewCount + 1		
KB-014	Agent attaches KB article to ticket comment	POSITIVE	Medium
Steps	1. Login as Agent 2. POST /api/tickets/{id}/comments with kbArticleId reference 3. Expect 201 4. Verify KB link appears in comment		
Expected	Expected: Comment saved with KB article link		
KB-015	Ticket marked Resolved via KB with article reference	POSITIVE	Medium
Steps	1. Agent resolves ticket 2. PATCH /api/tickets/{id}/resolve body={resolvedViaKb:true, kbArticleId:...} 3. Expect 200 4. Verify ticket has kbArticleId and resolvedViaKb=true		
Expected	Expected: Ticket marked with KB resolution reference		
KB-016	Article deletion logged in audit trail	POSITIVE	High
Steps	1. Admin deletes a KB article 2. Query audit log 3. Verify action=KB_ARTICLE_DELETED with articleId		
Expected	Expected: Audit entry KB_ARTICLE_DELETED		
KB-017	Create article without required Title returns 400	NEGATIVE	Medium
Steps	1. POST /api/kb/articles without title 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — title required		
KB-018	Version history visible to Admin and original author only	SECURITY	Medium
Steps	1. Agent creates article 2. Login as different Agent 3. GET /api/kb/articles/{id}/versions 4. Expect 403		
Expected	Expected: 403 for non-owner non-admin		

ESC — Escalation Workflow

17 test cases

Covers manual escalation, auto-escalation triggers, priority bumping, acknowledgement flow, and edge cases like escalating already-escalated tickets.

ESC-001	Agent escalates assigned ticket with mandatory reason	POSITIVE	Critical
Steps	1. Login as Agent 2. POST /api/tickets/{id}/escalate body={reason:"Customer is blocked, urgent fix needed"} 3. Expect 200 OK 4. Verify isEscalated=true on ticket		
Expected	Expected: 200 OK — escalation flag set, reason saved		
ESC-002	Admin escalates any ticket	POSITIVE	Critical
Steps	1. Login as Admin 2. POST /api/tickets/{id}/escalate with reason 3. Expect 200 OK		
Expected	Expected: 200 OK — escalation flag set		
ESC-003	Escalation without reason returns 400	NEGATIVE	High
Steps	1. POST /api/tickets/{id}/escalate with empty reason 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — reason is required		
ESC-004	Escalation reason exceeding 500 chars returns 400	NEGATIVE	Medium
Steps	1. POST /api/tickets/{id}/escalate with 501-char reason 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — max 500 chars		
ESC-005	Priority auto-raises one level on escalation	POSITIVE	High
Steps	1. Create Medium ticket 2. Escalate ticket 3. Verify priority changed to High		
Expected	Expected: Priority bumped: Low→Medium, Medium→High, High→Critical		
ESC-006	Critical priority stays Critical after escalation	EDGE CASE	High
Steps	1. Create Critical priority ticket 2. Escalate ticket 3. Verify priority remains Critical (no change)		
Expected	Expected: Priority unchanged — already at maximum		

ESC-007	Escalated ticket appears at top of Admin list	POSITIVE	High
Steps	1. Create and escalate several tickets 2. Login as Admin 3. GET /api/tickets 4. Verify escalated tickets sorted at top by escalation time		
Expected	Expected: Escalated tickets sorted first by escalationTime		
ESC-008	Admin acknowledges escalation	POSITIVE	High
Steps	1. Admin calls PATCH /api/tickets/{id}/escalation/acknowledge 2. Verify acknowledgementTimestamp recorded 3. Verify ticket no longer highlighted as unacknowledged 4. Verify isEscalated flag still true		
Expected	Expected: Acknowledged: timestamp set, highlight removed, flag preserved		
ESC-009	Cannot re-escalate in same status cycle	EDGE CASE	High
Steps	1. Escalate ticket (status=InProgress) 2. Attempt to escalate again while still InProgress 3. Expect 409 Conflict or 400		
Expected	Expected: 400/409 — already escalated in this status cycle		
ESC-010	After status change, ticket can be re-escalated	EDGE CASE	Medium
Steps	1. Escalate ticket in InProgress 2. Move ticket to OnHold (new status cycle) 3. Escalate again — expect 200		
Expected	Expected: 200 OK — re-escalation allowed after status change		
ESC-011	Critical ticket auto-escalated if unassigned after 30 min	POSITIVE	Critical
Steps	1. Create Critical ticket 2. Do NOT assign it 3. Wait 30 minutes (or fast-forward time) 4. Verify auto-escalation flag set		
Expected	Expected: isEscalated=true with escalationSource=SystemAutoEscalation		
ESC-012	SLA breach triggers auto-escalation	POSITIVE	Critical
Steps	1. Create ticket 2. Allow SLA to breach without closure 3. Verify auto-escalation triggered at breach point		
Expected	Expected: Auto-escalation at SLA breach, labelled as system		
ESC-013	OnHold ticket auto-escalated after 3 business days	POSITIVE	High
Steps	1. Move ticket to OnHold 2. Add no comments or status changes for 3 business days 3. Verify auto-escalation triggered		
Expected	Expected: Auto-escalation after 3 business days on hold		

ESC-014	Auto-escalation labelled System Auto-Escalation	POSITIVE	High
Steps	1. Trigger any auto-escalation condition 2. Check escalation reason and audit log		
Expected	Expected: reason='System Auto-Escalation', audit labelled accordingly		
ESC-015	User reopens closed ticket 3+ times triggers auto-escalation	EDGE CASE	High
Steps	1. User reopens same ticket 3 times 2. After 3rd reopen verify isEscalated=true, escalationSource=System		
Expected	Expected: Auto-escalation flag set on 3rd+ reopen by user		
ESC-016	Ticket escalated then immediately closed — flag preserved	EDGE CASE	Medium
Steps	1. Escalate ticket 2. Admin immediately closes ticket 3. Fetch closed ticket 4. Verify isEscalated=true still on closed record		
Expected	Expected: Escalation flag permanently on record even after closure		
ESC-017	Auto-escalation on already-escalated ticket (same cycle)	EDGE CASE	Medium
Steps	1. Manually escalate ticket 2. Trigger auto-escalation condition (e.g. SLA breach) 3. Verify no duplicate escalation, no error thrown		
Expected	Expected: Second escalation silently ignored; first record preserved		

Covers survey delivery, expiry enforcement, duplicate submission prevention, visibility rules, and opt-out behaviour.

CST-001	CSAT survey link emailed on ticket closure	POSITIVE	High
Steps	1. Admin closes ticket 2. Check User inbox 3. Verify survey link in email 4. Verify link is unique per ticket		
Expected	Expected: Survey email with unique link received by User		
CST-002	Survey visible on closed ticket page for 7 days	POSITIVE	High
Steps	1. Admin closes ticket 2. Login as User, fetch ticket detail 3. Verify survey link visible 4. After 7 days, verify link no longer shown		
Expected	Expected: Survey link visible ≤7 days post-closure		
CST-003	Survey link expires after 7 days	NEGATIVE	High
Steps	1. Close ticket 2. Wait 7+ days (fast-forward) 3. POST /api/csats/{ticketId} with survey response 4. Expect 410 Gone or 400		
Expected	Expected: 400/410 — survey link expired		
CST-004	User submits valid survey rating 1-5	POSITIVE	High
Steps	1. POST /api/csats/{ticketId} body={rating:4, comment:'Good service'} 2. Expect 200 OK 3. Verify response saved with ticket, agent, and category		
Expected	Expected: 200 OK — survey saved		
CST-005	Rating outside 1-5 returns 400	NEGATIVE	Medium
Steps	1. POST /api/csats/{ticketId} body={rating:6} 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — rating must be 1-5		
CST-006	Duplicate survey submission rejected	NEGATIVE	High
Steps	1. Submit CSAT survey for ticket 2. Submit again for same ticket 3. Expect 409 Conflict		
Expected	Expected: 409 Conflict — survey already submitted		

CST-007	Comment exceeding 500 chars returns 400	NEGATIVE	Medium
Steps	1. POST /api/csat with comment = 501 chars 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — comment max 500 chars		
CST-008	Survey associated with ticket, agent, category	POSITIVE	High
Steps	1. Submit CSAT for ticket closed by Agent A 2. Verify survey record links to ticket ID, agentId=A, categoryId		
Expected	Expected: Survey correctly associated with all three entities		
CST-009	Agent views only aggregate CSAT score	POSITIVE	High
Steps	1. Login as Agent 2. GET /api/agents/me/csat 3. Verify response shows aggregate score — no individual comments		
Expected	Expected: Aggregate score only, no individual comments		
CST-010	Individual survey responses visible only to Admin	SECURITY	High
Steps	1. Login as Admin 2. GET /api/csat/responses 3. Verify individual responses with comments accessible 4. Login as Agent — same endpoint — expect 403		
Expected	Expected: Admin: 200 with responses; Agent: 403		
CST-011	No CSAT score shown if fewer than 5 responses	EDGE CASE	Medium
Steps	1. Agent has 3 survey responses 2. GET /api/agents/{id}/csat from Admin view 3. Verify score field is null or hidden		
Expected	Expected: CSAT score not displayed — insufficient data		
CST-012	User opts out of survey emails	POSITIVE	Medium
Steps	1. User disables CSAT survey emails in notification preferences 2. Admin closes user's ticket 3. Verify user does NOT receive survey email		
Expected	Expected: No survey email for opted-out user		
CST-013	Opt-out applies to future closures only	EDGE CASE	Medium
Steps	1. Admin closes ticket (survey email already sent) 2. User opts out after receiving email 3. Verify existing link still valid (not revoked)		
Expected	Expected: Existing sent link still usable; future emails suppressed		

CST-014	CSAT submission logged in audit trail	POSITIVE	Medium
Steps	1. User submits CSAT survey 2. Query audit log 3. Verify action=CSAT_SUBMITTED with ticketId and userId		
Expected	Expected: Audit entry CSAT_SUBMITTED		

DEPT — Department Management

14 test cases

Validates department CRUD, protection of General department, active-user constraint on deactivation, and Department Head permissions.

DPT-001	Admin creates new department	POSITIVE	High
Steps	1. Login as Admin 2. POST /api/departments body={name:'Engineering', departmentHeadId:...} 3. Expect 201 Created 4. Verify department active and in list		
Expected	Expected: 201 Created — new department		
DPT-002	Duplicate department name returns 400	NEGATIVE	High
Steps	1. Create department 'Finance' 2. POST /api/departments with name='Finance' again 3. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — name must be unique		
DPT-003	Department name >100 chars returns 400	NEGATIVE	Medium
Steps	1. POST /api/departments with 101-char name 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — max 100 chars		
DPT-004	Admin renames department	POSITIVE	Medium
Steps	1. PATCH /api/departments/{id} body={name:'NewName'} 2. Expect 200 OK 3. Verify all existing tickets from that dept show new name		
Expected	Expected: 200 OK — rename reflected immediately		
DPT-005	Admin deactivates department with no active users	POSITIVE	High
Steps	1. Ensure no active users in dept 2. PATCH /api/departments/{id} body={status:Inactive} 3. Expect 200 OK		
Expected	Expected: 200 OK — department deactivated		
DPT-006	Deactivate department with active users returns 400	NEGATIVE	High
Steps	1. Assign active users to department 2. Attempt to deactivate 3. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — cannot deactivate with active users		

DPT-007	Delete department returns 405	NEGATIVE	High
Steps	1. DELETE /api/departments/{id} 2. Expect 405 Method Not Allowed		
Expected	Expected: 405 Method Not Allowed — deletion not permitted		
DPT-008	'General' department cannot be deactivated	EDGE CASE	Critical
Steps	1. Attempt PATCH /api/departments/general body={status:Inactive} 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — General dept is protected		
DPT-009	Admin changes Department Head	POSITIVE	High
Steps	1. Assign User A as Department Head 2. Change to User B 3. Verify User A no longer receives dept escalation emails 4. Verify User B now receives them		
Expected	Expected: Head changed; notification routing updated immediately		
DPT-010	Moving user between departments does not affect prior tickets	POSITIVE	Medium
Steps	1. User in Dept A raises ticket 2. Move user to Dept B 3. Verify old ticket still shows Dept A		
Expected	Expected: Historical ticket department unchanged		
DPT-011	Department summary returns correct metrics	POSITIVE	Medium
Steps	1. GET /api/departments/{id}/summary 2. Verify response contains activeUserCount, openTicketCount, ticketsLast30Days		
Expected	Expected: 200 OK with correct metrics		
DPT-012	One user can head multiple departments	POSITIVE	Medium
Steps	1. Assign same User A as head of Dept X and Dept Y 2. Verify User A receives escalation emails from both depts		
Expected	Expected: User A gets alerts for both departments		
DPT-013	Department Head has view-only access to dept tickets	POSITIVE	High
Steps	1. Assign Dept Head 2. Dept Head calls GET /api/tickets?department=A — expect 200 3. Dept Head calls PATCH ticket status — expect 403		
Expected	Expected: GET: 200; PATCH: 403		

DPT-014	Department creation logged in audit trail	POSITIVE	Medium
Steps	1. Admin creates department 2. Query audit log 3. Verify action=DEPARTMENT_CREATED		
Expected	Expected: Audit entry DEPARTMENT_CREATED		

RECUR — Recurring Ticket Templates

15 test cases

Covers template CRUD, schedule execution, business-day deferral, run logging, and edge cases like deactivated pre-assigned agents.

REC-001	Admin creates Daily recurring template	POSITIVE	High
Steps	1. Login as Admin 2. POST /api/recurring-templates with recurrencePattern=Daily, startDate=today 3. Expect 201 Created 4. Verify template active		
Expected	Expected: 201 Created — daily template scheduled		
REC-002	Admin creates Weekly recurring template	POSITIVE	High
Steps	1. POST /api/recurring-templates with recurrencePattern=Weekly 2. Expect 201 Created		
Expected	Expected: 201 Created — weekly template		
REC-003	Admin creates Monthly recurring template	POSITIVE	Medium
Steps	1. POST /api/recurring-templates with recurrencePattern=Monthly 2. Expect 201 Created		
Expected	Expected: 201 Created — monthly template		
REC-004	Admin creates template with CRON expression	POSITIVE	Medium
Steps	1. POST /api/recurring-templates with recurrencePattern=Custom, cronExpression='0 9 1 * *' 2. Expect 201 Created		
Expected	Expected: 201 Created — custom CRON template		
REC-005	Template fires on non-business day — deferred	EDGE CASE	High
Steps	1. Create Weekly template that fires on Saturday 2. Verify ticket generated on the following Monday instead 3. Verify run log shows scheduledTime=Saturday, actualTime=Monday		
Expected	Expected: Ticket generated on next business day; run log accurate		
REC-006	Template fires on public holiday — deferred	EDGE CASE	High
Steps	1. Add a public holiday 2. Create template scheduled to fire on that holiday 3. Verify ticket generated on next business day		
Expected	Expected: Holiday skipped; next business day used		

REC-007	Generated ticket follows all normal rules	POSITIVE	High
Steps	1. Let recurring template fire 2. Verify generated ticket has SLA deadline set 3. Verify creation notification sent 4. Verify audit entry created		
Expected	Expected: All rules (SLA, notifications, audit) applied normally		
REC-008	Each template run logged correctly	POSITIVE	High
Steps	1. Let template fire 2. GET /api/recurring-templates/{id}/run-log 3. Verify entry with templateName, generatedTicketId, scheduledFireTime, actualFireTime		
Expected	Expected: Run log entry with all 4 fields		
REC-009	Admin manually triggers template run	POSITIVE	Medium
Steps	1. POST /api/recurring-templates/{id}/trigger 2. Expect 201 Created for new ticket 3. Verify run log entry with manualTrigger=true		
Expected	Expected: Ticket created immediately; logged as manual trigger		
REC-010	Paused template does not generate tickets	POSITIVE	High
Steps	1. Pause active template 2. Wait for scheduled fire time 3. Verify no new ticket generated		
Expected	Expected: No ticket generated while paused		
REC-011	Deleting template does not delete prior tickets	POSITIVE	High
Steps	1. Create template, let it fire (ticket T1 created) 2. DELETE /api/recurring-templates/{id} 3. Verify T1 still exists in system		
Expected	Expected: T1 intact after template deletion		
REC-012	Agent cannot create recurring template	SECURITY	High
Steps	1. Login as Agent 2. POST /api/recurring-templates 3. Expect 403		
Expected	Expected: 403 Forbidden		
REC-013	Pre-assigned agent is deactivated — template fires gracefully	EDGE CASE	Critical
Steps	1. Create template with assignedToId = Agent A 2. Deactivate Agent A 3. Let template fire 4. Verify ticket created (unassigned or handled) 5. Verify no system crash or unhandled error		
Expected	Expected: Ticket created unassigned; error/warning logged		

REC-014	Template creation without required fields returns 400	NEGATIVE	Medium
Steps	1. POST /api/recurring-templates without title 2. Expect 400 Bad Request		
Expected	Expected: 400 Bad Request — title required		

REC-015	Template with End Date stops generating after date	POSITIVE	High
Steps	1. Create template with endDate = next Monday 2. Let it fire on Monday 3. Let scheduled time pass on Tuesday 4. Verify no ticket on Tuesday		
Expected	Expected: No ticket generated after end date		

Covers all 9 report types, filter accuracy, export formats, async large-export handling, and role-scoped access.

RPT-001	Admin generates Ticket Volume Report	POSITIVE	High
Steps	1. GET /api/reports/ticket-volume?from=2025-01-01&to;=2025-01-31 2. Expect 200 OK 3. Verify totals match actual ticket count in DB for range		
Expected	Expected: 200 OK with accurate volume data		
RPT-002	Admin generates Agent Performance Report	POSITIVE	High
Steps	1. GET /api/reports/agent-performance?agentId=...&from;=... 2. Verify response includes ticketsResolved, avgResolutionTime, slaCompliancePct, avgFirstResponseTime		
Expected	Expected: 200 OK with per-agent metrics		
RPT-003	Admin generates SLA Compliance Report	POSITIVE	High
Steps	1. GET /api/reports/sla-compliance 2. Verify breakdown by priority and category 3. Verify percentages calculated correctly		
Expected	Expected: 200 OK with SLA compliance breakdown		
RPT-004	Admin generates Open Tickets Ageing Report	POSITIVE	High
Steps	1. GET /api/reports/ageing 2. Verify sorted by time open (oldest first) 3. Verify tickets >5 business days flagged		
Expected	Expected: Sorted ageing report with >5-day flags		
RPT-005	Admin generates Department Load Report	POSITIVE	Medium
Steps	1. GET /api/reports/department-load 2. Verify ticket counts per department 3. Verify correct for selected date range		
Expected	Expected: 200 OK with per-department counts		
RPT-006	Admin generates Escalation Report	POSITIVE	High
Steps	1. GET /api/reports/escalations 2. Verify includes escalation reason, time-to-escalation, resolution outcome, SLA impact		
Expected	Expected: 200 OK with all escalation fields		
RPT-007	Admin generates User Satisfaction Report	POSITIVE	Medium
Steps	1. GET /api/reports/csat 2. Verify avg CSAT per agent, per category, and overall		
Expected	Expected: 200 OK with CSAT averages		

RPT-008	Admin generates KB Usage Report	POSITIVE	Medium
Steps	1. GET /api/reports/kb-usage 2. Verify most-viewed articles, articles linked to resolved tickets, zero-view articles		
Expected	Expected: 200 OK with KB usage breakdown		
RPT-009	Agent views own personal stats	POSITIVE	High
Steps	1. Login as Agent 2. GET /api/reports/my-stats 3. Verify response contains only that agent's data		
Expected	Expected: 200 OK — personal stats only		
RPT-010	Agent cannot view another agent's stats	SECURITY	High
Steps	1. Login as Agent A 2. GET /api/reports/agent-performance?agentId=Agent B 3. Expect 403		
Expected	Expected: 403 Forbidden		
RPT-011	CSV export has UTF-8 encoding and header row	POSITIVE	High
Steps	1. GET /api/reports/ticket-volume/export?format=csv 2. Save file 3. Open in editor — verify UTF-8 encoding and header row on line 1		
Expected	Expected: UTF-8 CSV with column headers		
RPT-012	PDF export has branded header	POSITIVE	Medium
Steps	1. GET /api/reports/sla-compliance/export?format=pdf 2. Open PDF 3. Verify company logo/header at top		
Expected	Expected: PDF with branded header template		
RPT-013	Report >10,000 rows processed asynchronously	EDGE CASE	High
Steps	1. Trigger export for date range that yields >10,000 rows 2. Verify API returns 202 Accepted immediately 3. Verify email with download link sent when export ready		
Expected	Expected: 202 Accepted; download link emailed		
RPT-014	Async export link expires after 24 hours	EDGE CASE	Medium
Steps	1. Receive async export download link 2. Wait 24+ hours 3. Attempt to download 4. Expect 410 Gone or 403		
Expected	Expected: 410/403 — link expired after 24 hours		

RPT-015	Report generation logged in audit trail	POSITIVE	Medium
Steps	1. Admin generates any report 2. Query audit log 3. Verify action=REPORT_GENERATED with report type and filter params		
Expected	Expected: Audit entry REPORT_GENERATED with parameters		

RPT-016	Date preset 'Last 3 Months' returns correct window	POSITIVE	Medium
Steps	1. GET /api/reports/ticket-volume?preset=last3months 2. Verify fromDate = today - 90 days, toDate = today 3. Verify data matches		
Expected	Expected: Correct 90-day window applied		

Covers general settings, category management, user CSV import, data archival policy, notification config, and SMTP setup.

ADM-001	Admin updates System Name	POSITIVE	Medium
Steps	1. Login as Admin 2. PUT /api/settings body={systemName:'Acme IT Help Desk'} 3. Expect 200 4. Verify header and email templates reflect new name		
Expected	Expected: 200 OK — name updated across UI and emails		
ADM-002	Admin uploads Company Logo	POSITIVE	Low
Steps	1. POST /api/settings/logo with image file 2. Expect 200 OK 3. Verify logo appears in email templates and login page		
Expected	Expected: Logo updated in all templates		
ADM-003	Admin updates Support Email Address	POSITIVE	High
Steps	1. PUT /api/settings body={supportEmail:'helpdesk@acme.com'} 2. Trigger any notification email 3. Verify From: = helpdesk@acme.com		
Expected	Expected: All notifications sent from configured email		
ADM-004	Admin adds new ticket category	POSITIVE	High
Steps	1. POST /api/categories body={name:'Cybersecurity', description:'...'} 2. Expect 201 Created 3. Verify category appears in ticket creation form		
Expected	Expected: 201 Created — new category available		
ADM-005	Admin renames existing category	POSITIVE	High
Steps	1. PATCH /api/categories/{id} body={name:'IT Security'} 2. Expect 200 OK 3. Fetch existing tickets with old category 4. Verify they now show 'IT Security'		
Expected	Expected: Rename reflected immediately on all existing tickets		
ADM-006	Admin deactivates category	POSITIVE	High
Steps	1. PATCH /api/categories/{id} body={status:Inactive} 2. Verify category removed from ticket creation form 3. Verify existing tickets still show deactivated category name		
Expected	Expected: Category removed from form; historical tickets unaffected		

ADM-007	Delete category returns 405	NEGATIVE	High
Steps	1. DELETE /api/categories/{id} 2. Expect 405 Method Not Allowed		
Expected	Expected: 405 Method Not Allowed		
ADM-008	Admin sends test email from SMTP settings	POSITIVE	Medium
Steps	1. POST /api/settings/email/test body={recipient:'test@acme.com'} 2. Expect 200 OK 3. Verify test email received		
Expected	Expected: 200 OK and test email delivered		
ADM-009	Bulk import valid CSV creates accounts	POSITIVE	High
Steps	1. Prepare CSV with 5 valid rows: Name,Email,Department,Role 2. POST /api/users/import with CSV file 3. Expect 200 OK 4. Verify 5 accounts created 5. Verify welcome email sent to each		
Expected	Expected: 5 accounts created; welcome emails sent; summary report returned		
ADM-010	CSV import with duplicate email skips row	NEGATIVE	High
Steps	1. Include email already in system in CSV 2. POST /api/users/import 3. Verify duplicate row listed as error 4. Verify existing account NOT overwritten		
Expected	Expected: Duplicate skipped and listed as error		
ADM-011	CSV import with invalid rows lists all errors first	NEGATIVE	High
Steps	1. Prepare CSV with 2 valid and 1 invalid row (missing email) 2. POST /api/users/import 3. Expect 400 with all errors listed 4. Verify NO accounts created until errors fixed		
Expected	Expected: 400 — all errors listed; no accounts created		
ADM-012	Admin receives import summary report	POSITIVE	Medium
Steps	1. Complete CSV import with mix of success, skip, error 2. Verify response body contains: accountsCreated, rowsSkipped, errors list		
Expected	Expected: Summary: created count, skipped count, error details		
ADM-013	Admin configures data archival policy	POSITIVE	Medium
Steps	1. PUT /api/settings body={archivalMonths:24} 2. Wait for archival job to run (or trigger manually) 3. Verify tickets closed >24 months ago have status=Archived		
Expected	Expected: Old tickets archived; unaffected by shorter range		

ADM-014	Archived tickets are read-only	SECURITY	High
Steps	1. Identify archived ticket 2. Attempt PATCH status 3. Expect 403		
Expected	Expected: 403 Forbidden — archived ticket is read-only		
ADM-015	Archived tickets excluded from default list	POSITIVE	High
Steps	1. GET /api/tickets (no filter) 2. Verify archived tickets not in results 3. GET /api/tickets?includeArchived=true 4. Verify archived tickets now included		
Expected	Expected: Archived excluded by default; visible with filter		
ADM-016	Permanent ticket deletion returns 405	NEGATIVE	Critical
Steps	1. DELETE /api/tickets/{id} 2. Expect 405 Method Not Allowed		
Expected	Expected: 405 Method Not Allowed — deletion not allowed		
ADM-017	All settings changes logged in audit trail	POSITIVE	High
Steps	1. Admin changes SLA targets, business hours, and system name 2. Query audit log 3. Verify SETTINGS_CHANGED entries for each change with old/new values		
Expected	Expected: All setting changes in audit log		
ADM-018	Session timeout configurable 15 min to 8 hours	POSITIVE	Medium
Steps	1. PUT /api/settings body={sessionTimeoutMinutes:60} 2. Expect 200 OK		
Expected	Expected: 200 OK — session timeout updated		
ADM-019	Session timeout outside range returns 400	NEGATIVE	Medium
Steps	1. PUT /api/settings body={sessionTimeoutMinutes:5} (below 15 min) 2. Expect 400 Bad Request 3. Try 500 min (above 480 min) — expect 400		
Expected	Expected: 400 Bad Request — out of allowed range		